

Secure By Design: An Exploration of the Application of Generative AI in Threat Modeling Technical Design Documents

Author: Mark Oswald, mark.james.oswald@gmail.com

Advisor: Lenny Zeltser

Accepted: April 26, 2026

Abstract

Threat modeling is a crucial component of secure software development but is a manual and time-intensive process that demands specialized security expertise. This paper explores the efficacy of large language models (LLMs) for creating comprehensive threat models by analyzing technical design documents, particularly when provided with additional contextual information about the product's underlying infrastructure and deployment environment. LLMs from multiple providers are given a common technical design document to analyze and generate a threat model, with the resulting outputs compared against a baseline threat model. Each model is rated on the accuracy and relevance of its results (relative to the baseline threat model). Each model is also assessed on its ability to identify legitimate, architecturally significant novel threat scenarios that were not included in the original baseline. One-third of the trials use a hybrid approach that combines multiple models. The findings indicate the viability of using LLMs to augment an enterprise's security posture by enabling developers to quickly, efficiently, and effectively conduct threat modeling at scale.

1. Introduction

The use of LLMs is now common in the technology sector. In a 2025 Q1 survey of nearly 2,000 participants with software development and engineering backgrounds, 91% reported using an LLM at work. Also, 74% said LLMs made them more productive than before (Sleegers et al., 2026). AI is increasingly used in security for threat detection, vulnerability assessment, incident response, and automation (Ferrag et al., 2025; Mirtaheri et al., 2025; Zhang et al., 2025).

The Threat Modeling Manifesto defines *threat modeling* as “analyzing representations of a system to highlight concerns about security and privacy characteristics” (Braiterman et al., 2020). It is a critical cybersecurity process that identifies potential threats and suggests mitigating actions and controls to reduce risks for system designs (Wu et al., 2025; Kaviyil et al., 2023). The process lets a developer identify key components, data flows, and trust boundaries in their application. This is sometimes called “decomposing the application.” The process also helps to identify threat targets and entry points that attackers could use (Conklin, n.d.).

Threat modeling is an important piece of an organization's security review or assessment process and is a common prerequisite before a new software design is deployed to a production environment (Al-Hashimi et al., 2025). However, it is a labor-intensive process that can require considerable time, effort, and expertise to conduct effectively. As it is largely manual, it can be prone to inefficiencies and human error when completing subtasks such as creating data flow diagrams, identifying threats, and mapping to actionable mitigations (Wu et al., 2025). Per Yskout et al. (2020), the threat modeling process can take upwards of 45 hours to complete, even when conducted by threat modeling experts, where all stakeholders are aware of the project's requirements. Given the context of these resource demands, according to one survey, only half of businesses perform formal risk evaluations during software development, with even fewer (30%) including security testing in the initial phases (Alzahrani & Khan, 2024).

Several studies have examined the use of LLMs in threat modeling. Kaviyil et al. (2023) conducted research into the use of ChatGPT to generate a threat model for a

microservice hosted on AWS. They found that while the model could identify relevant threats and suggest mitigations, LLM-generated outputs were better used as a complement to human judgment and traditional threat modeling methodologies. Wu et al. (2025) introduced ThreatModeling-LLM, a framework combining prompt engineering techniques and model fine-tuning to automate threat modeling for banking systems, demonstrating marked improvements in threat identification accuracy. AWS Cloud Security Engineering (ACSE)-Eval, a dataset containing 100 production-grade AWS deployment scenarios, was used to evaluate several LLMs' threat modeling capabilities against real-world cloud deployment infrastructure (Munshi et al., 2025). Tools such as STRIDE GPT aim to make comprehensive threat modeling more accessible by automating and simplifying the process (especially for developers without deep security expertise), leveraging modern LLMs to complete the process end-to-end (Raghavan, 2025).

Whether through individual model evaluation, domain-specific fine-tuning, or the creation of benchmark datasets, each of these studies has demonstrated the potential of LLM-assisted threat modeling from different angles. Taken together, this body of work highlights the promise of LLM-assisted threat modeling, but also reveals a lack of unified, comparative evaluation across approaches. This study combines several of these dimensions into a single comparative framework: multiple commercial LLMs from different providers are evaluated across repeated trials on a common threat modeling task, supplemented with contextual grounding data, and assessed not only for baseline threat coverage and novel threat discovery but also for severity calibration quality and output consistency. Through the analysis of empirical data gathered through multiple trials of autonomous threat modeling conducted by different modern LLMs, this study tests the following hypothesis: given meticulously crafted prompts and sufficient contextual data describing the deployment environment, underlying infrastructure, and security controls already in place, an LLM can generate a threat model with a high degree of threat coverage. Beyond identifying known threats, the LLM can also identify legitimate, architecturally significant novel threat scenarios that are not present in a baseline threat model created by a human security professional.

2. Research Method

This study uses the STRIDE threat modeling method, as outlined by the Open Worldwide Application Security Project (OWASP), which identifies threat scenarios in six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege (Conklin). LLMs from multiple providers analyzed the same technical design document to generate STRIDE threat models. The resulting threat models were compared with a baseline threat model created for the document beforehand to assess threat coverage. For each trial, LLMs were given access to contextual data about the underlying infrastructure and security controls already in place in the deployment environment to ground their output via retrieval-augmented generation (RAG). Threat models were output in comma-separated values (CSV) format. All prompts used, the design document assessed, the baseline threat model, and the grounding data can be found in the Appendix.

2.1. STRIDE Methodology

The STRIDE threat modeling process was chosen as it is a practical, simple, and well-documented method of identifying and categorizing threats, emphasizing the identification and classification of key components and data flows within an architecture (Conklin). Each LLM followed this iterative process to create, reconcile, and finalize every threat model:

1. The LLM analyzes the design document, identifies keywords and search terms, and builds a bundle of between five and ten single-line queries that will be used for grounding data retrieval.
2. The LLM analyzes the design document and identifies key components, trust boundaries, and data flows.
3. With these key details identified, the LLM uses the query bundle to search for and retrieve grounding data that is relevant to the design document. With this additional contextual data in hand, the LLM generates a threat model.

4. The LLM analyzes the threat model it has just generated, then undergoes a revision and reconciliation process to make corrections and adjustments before outputting the final draft.

Prompts were created using Salesforce Prompt Builder, a tool available on the Salesforce platform that allows users to create prompt templates that generate content using LLMs (Salesforce, 2024). Apex code was called directly in the prompt templates to perform a vector-based search in Salesforce Data Cloud for grounding data. Data Cloud (also known as Data 360) is a Salesforce-owned platform that can be used to host and unify data from various sources, allowing the processing and querying of data at scale (Salesforce, n.d.-c). Per Salesforce's *Apex Developer Guide*, Apex is a “strongly typed, object-oriented programming language that allows developers to execute flow and transactional control statements on the Salesforce Platform server, in conjunction with calls to the API” (Salesforce, n.d.-a). Prompts were chained together and called sequentially using a Salesforce Flow, a low-code tool for creating automated processes (Salesforce, n.d.-b).

LLMs were instructed to treat perceived gaps in the design document as threats. For example, if the LLM identified a key dataflow between two worker nodes, but there was either a poorly defined or undefined method of how mutual transport layer security (mTLS) would be implemented to secure communications between the two, the LLM would treat that as a potential threat and identify it as a gap in the threat model. Threat models were output in CSV format, with columns for threat category, threat scenario, affected components, risk rating, and mitigation guidance. More threats were identified in the system than were included in the baseline threat model, to assess each LLM's ability to identify legitimate, novel threats that a human threat modeler might have missed.

2.2. Grounded Technical Design Document Analysis

Each LLM analyzed a single common technical design document during the threat modeling process. The design document described an enterprise software application that processes customer orders, built on AWS infrastructure and incorporating on-premises/legacy architectural components. It was intentionally written to include

security flaws and gaps for the LLMs to identify and analyze. Each LLM also had access to contextual data about the deployment environment, the underlying infrastructure, and security control implementations, which could be used to ground their threat model outputs.

2.3. LLM Selection

The four LLMs selected for this research include OpenAI GPT-5, OpenAI GPT-5 Mini, Anthropic Claude 4.5 Sonnet, and Anthropic Claude 4.5 Haiku. Anthropic's Claude Opus, the provider's most advanced model, was unavailable to the researcher at the time of testing and was therefore excluded from this study. Each of these LLMs is multimodal, allowing them to process both text and image content when generating a threat model. This selection of LLMs includes a blend of cheaper, faster models (GPT-5 Mini and Claude 4.5 Haiku) and models capable of deeper logic and reasoning (GPT-5 and Claude 4.5 Sonnet). Each of these was also tested in a hybrid configuration in which the faster model (GPT-5 Mini and Claude 4.5 Haiku) was used to build query packs and create an initial threat model, while the deeper logic and reasoning models (GPT-5 and Claude 4.5 Sonnet) handled the reconciliation and revision iteration before outputting the final draft of the threat model.

3. Findings and Discussion

The following subsections present and explore the results of each model's trial set (6 total trial sets, with 10 trials run per set). Each trial set's results are summarized in a table that captures the trial number, severity breakdown of identified threats (Critical, High, Medium, or Low), the number of threats matched against the baseline threat model, the number of novel threats identified, the total combined number of baseline and novel threats identified, and the time it took for each trial to complete, in seconds. There were 16 threats in the baseline threat model; accordingly, a trial with 16 baseline threat matches would have 100% baseline threat coverage, a trial with 8 would have 50%, and so on. For this study, high threat coverage is defined as an LLM reproducing at least 60% of baseline threats on average across its trial set.

3.1. OpenAI Models

Three sets of 10 threat modeling trials were run with OpenAI GPT models. GPT-5 was used exclusively for the first trial set. GPT-5 Mini was used exclusively for the second trial set. A hybrid approach combining GPT-5 and GPT-5 Mini was used for the third and final OpenAI trial set.

3.1.1. GPT-5

Trial	Total	Crit	High	Med	Low	Baseline	Novel	Runtime
1	12	0	0	6	6	7	5	81.8s
2	18	0	0	1	17	10	8	80.4s
3	19	0	0	3	16	11	8	77.5s
4	12	0	0	9	3	8	4	57.9s
5	12	0	0	2	8	9	3	67.4s
6	17	0	0	11	6	9	8	83.2s
7	12	0	0	8	4	7	5	62.1s
8	12	0	1	4	7	6	6	75.3s
9	11	0	0	6	5	7	4	61.2s
10	12	0	1	8	3	9	3	62.5s
Avg	13.7	0.0	0.2	5.8	7.5	8.3	5.4	70.9s

Table 1: GPT-5 Trial Results

Not accounting for differences in risk ratings for threat scenarios that GPT-5 matched to the baseline threat model, GPT-5 averaged 8.3 shared threats per trial, for an overall average of 52% threat coverage, compared to the 16 threats identified in the baseline. GPT-5 was among the fastest-performing LLMs, averaging just under 71 seconds per trial for generating threat models. Seven of the trials (1, 4, 5, 7, 8, 9, 10) demonstrated high precision, with baseline threat matches accounting for the majority of findings.

By far the most significant pattern was GPT-5's risk reductions compared to the baseline threat model. Of the five critical findings in the baseline threat model, none were retained in any of the ten trial runs. Of the eight high findings in the baseline, only a single finding (Message Relay infinite retry, a denial-of-service threat) was retained in two of the trials (the eighth and tenth). GPT-5 appeared to be highly influenced by the available grounding data, reducing nearly every high- and critical-risk threat to low or moderate based on its knowledge and analysis of security controls in the deployment environment.

GPT-5 showed a sporadic tendency to identify novel threat scenarios that were not in the baseline threat model but were legitimate threats backed by evidence from both the design document and the grounding documents. Three of the trials (2, 3, 6) produced more total findings than the 16 threats in the baseline threat model. These were some of the more interesting and valid novel threats:

- Outbox payload integrity (Trials 2, 3): The design document states that outbox payloads are encrypted but does not specify whether they are signed. A compromised application session or DB-level attacker could modify ciphertext or swap payloads without detection.
- IAM database authentication gap (Trials 3, 6): The design document states that database security is "managed at the application layer," but it does not mention IAM database authentication. The grounding documents confirm RDS uses IAM database authentication in the production environment, but the design document does not specify it. This is a legitimate gap threat scenario that was missing from the baseline threat model.
- Feature flag and admin action audit trail (Trial 8): The design document describes a feature flag (**enable_async_order_flow**) for rollout/rollback but does not specify any change audit logging for it. A malicious insider toggling the flag could redirect traffic without accountability.

3.1.2. GPT-5 Mini

It is immediately noticeable that, despite GPT-5 Mini being billed as a faster, cheaper model, it actually took, on average, considerably longer to complete each trial run than standard GPT-5. GPT-5 Mini had a slightly higher average percentage of threat coverage than GPT-5 (10.0 baseline threats matched per threat model, 62%).

Trial	Total	Crit	High	Med	Low	Baseline	Novel	Runtime
1	13	0	0	3	10	9	4	69.5s
2	17	0	4	6	7	10	7	106.1s
3	12	0	0	5	7	9	3	88.2s
4	22	0	4	10	8	10	12	146.8s
5	25	1	0	10	14	12	13	150.2s
6	24	0	0	7	17	11	13	129.9s
7	17	1	0	7	9	10	7	95.0s
8	15	0	1	7	7	11	4	97.0s
9	19	0	3	9	7	11	8	103.0s
10	11	0	0	5	6	7	4	69.7s
Avg	17.5	0.2	1.2	6.9	9.2	10.0	7.5	105.5s

Table 2: GPT-5 Mini Trial Results

GPT-5 Mini often introduced novel threat scenarios not covered in the baseline threat model, with 60% of the trials having more threat scenarios than the baseline. The fifth trial topped out with 25 threat scenarios, which was more than double the number of threats identified in the tenth trial (11). This gap in identified scenarios is indicative of a varied and inconsistent threat identification scope, although some of the valid novel threats identified appeared in several trials, such as:

- Trace-to-principal correlation (Trials 4, 6, 9): The design document mentions X-Ray for tracing and CloudWatch for logging but does not map trace IDs to authenticated principals. The grounding data mentions a Cognito integration that

validates JWT claims at the gateway, and X-Ray propagates trace IDs. However, there is no documented mapping between the two. This represents a repudiation gap: while an investigator could observe an event, they would not be able to reliably determine who was responsible.

- Container image provenance and CI/CD artifact integrity (Trials 1, 9): This finding relates to a baseline scenario detailing an unhardened container threat. While the baseline focused on runtime hardening, this threat focused on supply chain security, highlighting the risks posed by unsigned images and the lack of explicit artifact signing in the CI/CD pipeline. Although the grounding data mandates SBOMs and hardened base images, the design document never references image signing or provenance verification, leaving this a valid gap in the baseline threat model.

Like GPT-5, GPT-5 Mini also showed a tendency to downgrade risk ratings based on context from the grounding data. It showed a greater willingness to retain some of the critical and high ratings, but most ratings for matching threats were reduced to moderate or low, based on its analysis of relevant grounding data.

3.1.3. Hybrid Approach with GPT-5 and GPT-5 Mini

For each trial, GPT-5 Mini was used to build the query pack and generate the initial threat model. GPT-5 was responsible for the reconciliation and revision phase, outputting the final draft of the threat model. This hybrid approach resulted in an average threat coverage of 59% per trial.

Trial	Total	Crit	High	Med	Low	Baseline	Novel	Runtime
1	24	0	3	11	10	11	13	122.2s
2	21	0	1	6	7	10	11	114.0s
3	12	0	0	7	5	9	3	103.2s
4	12	0	0	6	6	6	6	94.3s
5	26	0	0	17	9	9	17	141.3s

6	26	0	3	12	11	13	13	134.2s
7	14	0	2	5	7	10	4	93.6s
8	16	0	1	9	6	9	7	86.6s
9	18	0	1	6	11	9	9	109.5s
10	12	0	3	7	2	9	3	87.8s
Avg	18.1	0.0	1.4	8.6	7.4	9.5	8.6	108.7s

Table 3: GPT-5/GPT-5 Mini Hybrid Trial Results

However, as with the GPT-5 and GPT-5 Mini trials, multiple valid novel threat scenarios were identified that were not present in the baseline threat model. 50% of the trials resulted in threat models with more total findings than the baseline threat model. It should be noted that Trial 2 included multiple scenarios rated as having no risk at all, making it an outlier and somewhat straying from the instructions given in the prompts. On average, hybrid trials took longer to complete than trials using only one model.

There was one baseline threat (SQL injection) that was only identified in Trial 9. Another baseline finding (an unhardened container) was detected in only 4 of 10 trials. Two baseline threats (cloud-to-mainframe boundary and large payload congestion) were not matched in any of the hybrid trials. These trials also exhibited another hallmark of GPT-driven threat analysis: no critical risk ratings were present in any of the trials, and only a sparse offering of high-risk ratings, with three trials (Trials 3, 4, 5) having only moderate and low risk ratings.

It appeared that GPT (regardless of specific model) was eager, if not aggressive, to adjust risk ratings to lower severities, and it even seemed to make assumptions not directly supported by any of the design or grounding data in doing so. For example, a critical finding in the baseline scenario highlighted the storage of encryption keys in environment variables. This critical finding was consistently downgraded to moderate or low in the trials, likely because the grounding data mentioned the use of KMS and AWS Secrets Manager. Several of the trials stated that migrating from environment variables to KMS was a fact, even though there was no explicit confirmation in the grounding data

beyond the mandated use of KMS and AWS Secrets Manager. Many of the trial scenarios included language such as "gap resolved," "invalidated," and "new evidence confirms." This indicated that the model (in this case, GPT-5) assessed residual risk after controls were applied, rather than the original design flaws.

As with the GPT-5 Mini trials, there was a wide range of findings across trials, ranging from a minimum of 12 (Trials 3, 4, and 10) to a maximum of 26 (Trials 5 and 6). This does not appear to reflect a difference in perceived threat surfaces. Instead, it highlights inconsistencies in how the models used in hybrid trials break down threat scenarios into multiple detailed scenarios.

There was also a sizable time difference between trials, with the shortest (Trial 8) finishing in 86.58 seconds and the longest (Trial 5) taking 141.34 seconds. Longer trials do not necessarily correlate with an increase in threat coverage compared to the baseline, however. For example, Trial 7 (93.6 seconds) had 10 matching findings, compared to Trial 9 (109.5 seconds), which had 9. This suggests that the model produces diminishing returns relatively quickly, and that longer trial runs tend to yield more granular findings that do not consistently cover a larger portion of the technical design's threat surface in a meaningful way. Time does not guarantee quality.

3.2. Anthropic Models

Following the OpenAI GPT trials, three sets of 10 threat modeling trials were run with Anthropic models. Claude 4.5 Sonnet was used exclusively for the first Anthropic trial set. Claude 4.5 Haiku was used exclusively for the second trial set. A hybridized approach utilizing both Sonnet and Haiku was used for the third and final trial set.

3.2.1. Claude 4.5 Sonnet

Claude 4.5 Sonnet stands out compared to the GPT family because it produces a threat severity distribution that closely resembles the baseline threat model's distribution. The baseline threat model contained 16 threats (5 critical, 8 high, and 3 moderate).

Trial	Total	Crit	High	Med	Low	Baseline	Novel	Runtime
-------	-------	------	------	-----	-----	----------	-------	---------

1	20	4	4	11	1	11	9	356.1s
2	27	3	7	10	7	12	15	276.2s
3	19	4	3	9	3	10	9	388.2s
4	23	2	9	11	1	12	11	355.0s
5	22	0	1	9	12	12	10	265.0s
6	21	4	4	8	5	9	12	257.4s
7	24	3	5	9	7	13	11	179.8s
8	23	1	4	15	3	10	13	164.1s
9	17	2	5	10	0	8	9	306.9s
10	20	2	2	7	9	11	9	327.5s
Avg	21.6	2.5	4.4	9.9	4.8	10.8	10.8	287.6s

Table 4: Claude 4.5 Sonnet Trial Results

Claude 4.5 Sonnet averaged 2.5 critical findings and 4.4 high findings per trial, whereas there were only two total critical findings in 30 total GPT trials. Claude 4.5 Sonnet averaged 10.8 matched baseline threats per trial (68% average baseline threat coverage). Trial 9 (8 baseline matches) and Trial 6 (9 baseline matches) finished below average in threat coverage, while the remaining eight trials averaged 11.3 baseline threat matches (71%).

Claude 4.5 Sonnet also stands out in the amount of time it took for each trial to run. Only two trials (7 and 8) finished in under 3 minutes. There does not appear to be a strong correlation between time spent and baseline threat coverage. For example, Trial 3 was the slowest trial, with a runtime of 388.24 seconds, matching 10 baseline threats. Trial 8 was the fastest trial, with a runtime of 164.12 seconds, and also matched 10 baseline threats.

Sonnet's average baseline threat coverage is stronger than that of the GPT family. However, it also shares some identification blind spots: it missed the baseline cloud-to-mainframe boundary gap in every trial, and the SQL injection baseline threat in seven.

The baseline large-payload threat was missed in all 10 trials, and the unhardened container threat was missed in 3 trials. These results continue to indicate consistent blind spots across the LLMs. Sonnet is also superior in its tendency to rate threat severity more closely aligned with the baseline threat model, as well as in its identification of legitimate novel threats not mentioned in the baseline.

Sonnet generated more genuine novel threat scenarios than any of the GPT trials. These were some of the more common ones:

- Feature flag access control gap (identified in 9 trials): The same threat was identified in GPT-5 Trial 8, but Sonnet identified it much more consistently. It points out that, without a specified change audit log for the **enable_async_order_flow** rollback mechanism, it can be repurposed as an elevation-of-privilege vector for a malicious insider.
- Outbox table unbounded growth (identified in 9 trials): This was a unique and interesting finding. The design document defines outbox statuses, but this particular threat points out a design gap: there are no specified procedures for cleanup and archival. When considering the projected daily transaction volume (1.5 million) and growth (300%) defined in the design document, Sonnet identifies this as a potential future pathway for a denial-of-service threat.

3.2.2. Claude 4.5 Haiku

With an average runtime of 93.65, Claude 4.5 Haiku is the second-fastest model in this study, behind GPT-5. Its average threat coverage is 10.1 matched baseline threats per trial (63%).

Trial	Total	Crit	High	Med	Low	Baseline	Novel	Runtime
1	19	1	1	7	10	10	9	91.1s
2	18	1	2	6	9	11	7	95.1s
3	19	4	4	9	2	12	7	145.2s

4	15	0	1	6	3	9	6	83.2s
5	17	0	0	10	7	8	9	78.7s
6	16	0	0	6	10	11	5	87.5s
7	15	4	5	4	2	9	6	104.7s
8	14	3	1	9	1	10	4	81.5s
9	16	3	5	7	1	10	6	85.4s
10	17	0	2	5	0	11	6	84.0s
Avg	16.6	1.6	2.1	6.9	4.5	10.1	6.5	93.7s

Table 5: Claude 4.5 Haiku Trial Results

Although Haiku continues the Claude family trend of willingness to rate threats at high and critical levels, it still averages fewer findings of these severities than Sonnet. Observing the distribution of critical findings across trials reveals an interesting trend. In four of the trials (3, 7, 8, and 9), Haiku identified multiple critical threats, but in four other trials (4, 5, 6, and 10), Haiku failed to identify any. A single critical finding was identified in both Trial 1 and Trial 2. In these trials, Haiku seems to shift between weighing the flaws in the design document more heavily and treating them as lower risk, given the added context provided by the grounding data.

Like all of the other models in this study, Haiku fails to identify multiple baseline threats consistently:

- Cloud-to-mainframe boundary (missed in all ten trials)
- SQL injection (identified in three trials)
- Large payload congestion (missed in all ten trials)

Haiku also struggled to spot the unhardened container baseline threat, correctly identifying it in only 4 of 10 trials. Haiku generated multiple novel threat scenarios, including two (direct ALB access bypassing API gateway in Trial 3 and relay polling rate limiting in Trial 7) that were not identified by any other model in this study. It also

identified a supply chain threat scenario in Trial 2 that approaches the application's cryptographic implementation threat surface from a different angle than its predecessors. Like Sonnet, Haiku consistently identified novel threat scenarios involving feature-flag access control across a majority of its trials.

3.2.3. Hybrid Approach with Claude 4.5 Sonnet and Claude 4.5 Haiku

Trial	Total	Crit	High	Med	Low	Baseline	Novel	Runtime
1	23	3	4	12	4	9	14	151.9s
2	11	2	3	4	2	4	7	94.8s
3	15	3	7	5	0	11	4	169.5s
4	18	2	1	10	5	11	7	110.2s
5	13	3	4	6	0	7	6	119.0s
6	17	0	4	9	4	9	8	110.8s
7	16	2	7	7	0	7	9	128.6s
8	7	1	2	3	1	4	3	84.5s
9	18	0	1	6	11	11	7	117.4s
10	10	2	4	4	0	8	2	99.5s
Avg	14.8	1.8	3.7	6.6	2.7	8.1	6.7	118.6s

Table 6: Claude 4.5 Sonnet/Haiku Hybrid Trial Results

Like the GPT hybrid trials, these Claude-focused hybrid trials used the faster, cheaper model (Claude 4.5 Haiku) to build the query pack and generate an initial threat model. In contrast, the more expensive model (Claude 4.5 Sonnet) was used for reconciliation and revision to produce the final threat model. Baseline threat coverage was the weakest among the other trial groupings in the study, with an average of 8.1 baseline threat matches per trial (51%). This result appears skewed by some very low-performing trials - Trials 2 and 8 matched just four baseline threats apiece. The Claude hybrid approach is plagued by high variance and instability in identifying even

fundamental threats, which were picked up in the majority of other trials across all models. Interestingly, this variance and instability are not exhibited by either Sonnet or Haiku when they act alone.

Threat risk ratings per trial are relatively close to the same rate as the Sonnet trials, with the hybrid trials turning up 1.8 critical and 3.7 high findings per trial (Sonnet trials averaged 2.5 critical and 4.4 high findings per run). The hybridized approach tended to rate the vulnerabilities in the design document on their own merits, rather than automatically reducing risk based on security controls information in the grounding documents. The hybrid approach found at least one critical threat in 80% of its trials.

The Claude hybrid trials generated more novel threats than any other trial set. Although there was a diverse selection of novel threat concepts across these ten trials, many of them were not reproduced across multiple trials. The only novel threat scenario that showed significant reproducibility was the feature-flag auditability threat, which was identified in every other trial set in this study. It was found in seven of the Claude hybrid's ten trials. The hybrid trials did identify some interesting, if ultimately not reproducible, novel threats:

- AES-GCM authentication tag/bitflip attacks (Trial 2): This threat scenario examined the design document's AES-256-GCM cryptographic implementation and pointed out how it could be vulnerable to ciphertext manipulation if GCM authentication is not verified during decryption.
- SQL injection in outbox queries (Trial 7): This was actually an alternative take on the SQL injection baseline threat scenario that identified a novel threat vector. In this case, rather than sharing the baseline threat model's input sanitization concerns, it raises the possibility of dynamic query construction for outbox operations being a legitimate vulnerability.

3.3. Further Discussion

3.3.1. Comparative Analysis of All Six Trial Sets

The following table provides a quick view of the total number of baseline threats, the number of threats by severity, and the percentage of severities that comprise the baseline total.

Total Threats	Crit	High	Moderate	Low
16	5 (31%)	8 (50%)	3 (19%)	0 (0%)

Table 7: Baseline threat model's threat severity distribution

Table 8 (shown below) gives a side-by-side comparison of each trial set. For each trial set, it shows the average number of baseline threat matches, the average baseline threat coverage, the average severity distribution of matched baseline threats, the average number of novel threats identified, and the average runtime per trial (in seconds).

Model	Avg Baseline	Baseline Coverage	Baseline Severity Calibration	Avg Novel	Avg Runtime
GPT-5	8.3 / 16	52%	0% Crit, 2% High, 49% Moderate, 48% Low	5.4	70.9s
GPT-5 Mini	10.0 / 16	62%	2% Crit, 4% High, 39% Moderate, 55% Low	7.5	105.5s
GPT-5/GPT-5 Mini hybrid	9.5 / 16	59%	0% Crit, 4% High, 44% Moderate, 52% Low	8.6	108.7s

Claude 4.5 Sonnet	10.8 / 16	68%	18% Crit, 19% High, 36% Moderate, 28% Low	10.8	287.6s
Claude 4.5 Haiku	10.1 / 16	63%	18% Crit, 13% High, 37% Moderate, 31% Low	6.5	93.7s
Claude 4.5 Sonnet/Haiku hybrid	8.1 / 16	51%	21% Crit, 22% High, 36% Moderate, 21% Low	6.7	118.6s

Table 8: Comparison of all trial sets 'results

Generally speaking, baseline threat coverage averaged eight to eleven matches per trial across the 60 trials. Two baseline threats appeared to comprise something of a blind spot for every LLM:

- Cloud-to-mainframe boundary
- Large payload congestion

Each of these threats required models to essentially "think outside the box" and infer threats across design document boundaries by consistently identifying what was absent, rather than focusing on more obvious flaws. These two baseline threats were not identified in any of the sixty trials that comprised this study. Even current models may consistently struggle with this kind of negative pattern recognition, at least across these particular threat domains. Each model also consistently struggled to identify the SQL injection baseline threat scenario. The unhardened container baseline scenario also

stumped most models across most trials, though Sonnet found it with relatively high consistency (70% of trials).

Runtime costs were highly variable across the four models and two hybridized approaches assessed in this study. GPT-5 was the fastest LLM in the entire study, with an average trial runtime of 70.9 seconds, whereas Claude 4.5 Sonnet was by far the slowest, clocking in at 287.6 seconds on average, or nearly four times as long. Even compared to its Haiku sibling (with an average runtime of 93.7 seconds), Sonnet took roughly 3 times as long to complete the average trial.

Both model families diverged in their risk rating adjustments. Whereas the Claude family of models calibrated threat severities to be more in line with the baseline threat severities, the GPT family of models was eager to reduce risk based on information in the grounding data. In a real-world production environment, GPT's eagerness to reduce risk ratings would be problematic, as severity calibration is typically directly tied to prioritization of remediation efforts. By labeling the overwhelming majority of threats as moderate, low, or completely neutralized, GPT-5 and GPT-5 Mini provided less actionable output, even if they detected baseline threats at about the same level as the Claude 4.5 models.

There did not appear to be any significant gains from using a hybrid approach over a single model for threat model generation. In the case of the Claude hybrid trials, the variance in threat coverage proved too unreliable to use in a real production environment. However, it produced a wide and diverse set of novel threat scenarios across its ten trials. The GPT hybrid trials also failed to produce any notable gains in runtime cost or baseline threat coverage compared to the standalone GPT-5 and GPT-5 Mini trials. This lack of consistent output from either hybrid model approach would not bode well for their inclusion in a real-world automated threat modeling pipeline. It would quickly become quite expensive in terms of operational cost to have humans consistently reviewing these hybridized outputs to ensure stability. By contrast, the outputs generated by single models were consistently more stable, with fewer outlier trials skewing average results.

All models consistently found genuine novel threat scenarios not mentioned in the baseline threat model. This was indicative of the models' ability to reason beyond the threat surface presented in the baseline threat model when it analyzes both the design document's data flow and the added context provided by grounding data. This supports the paper's introduction, which asserts that LLMs can serve a valuable role in assisting humans in the threat modeling process. The LLMs in this study consistently identified legitimate vulnerabilities and gaps not mentioned in the baseline threat model.

3.3.2. Top Overall Pick: Claude 4.5 Haiku

Among the four models assessed in this study, Claude 4.5 Haiku ranks as the top overall pick when considering average runtimes and threat identification. With an average runtime of 93.7 seconds, it was the second-fastest model in the study, topped only by GPT-5 and far ahead of both GPT-5 Mini and Claude 4.5 Sonnet. In terms of baseline threat coverage, it averaged 10.1 baseline matches per trial, second only to Sonnet's 10.8. It identified an average of 6.5 novel threat scenarios per trial. It reproduced its top novel threat finding (feature flag access control) across 60% of its trials, indicating high consistency and stability in its threat analysis logic and reasoning. Haiku performed favorably in threat severity distribution as compared to the baseline, identifying an average of 1.6 critical and 2.1 high findings per trial.

3.3.3. Most Accurate and Relevant Output: Claude 4.5 Sonnet

Claude 4.5 Sonnet stands out as best in class for output quality, although it took the longest of the four models to generate it, with an average trial runtime of 287.6 seconds. It boasts the top baseline threat coverage in the study, averaging 10.8 baseline threat matches per run (68%). Where Sonnet really shines is its ability to generate legitimate novel threats with a high reproducibility rate, averaging 10.8 novel findings per trial. Its most frequent novel threat scenario (feature flag access control) occurred in 9 of its 10 trials, demonstrating remarkable analytical consistency. While its 21.6 average findings per trial point to excess noise and a potentially concerning rate of false positives, its severity distribution (2.50 critical and 4.40 high findings per trial) compares more favorably to the baseline than those of any other model in the study. It suggests that Sonnet is quite capable of identifying legitimate novel threat scenarios.

3.3.4. The Role of Grounding Data in Risk Rating Adjustment

One of the more important discoveries of this study concerns the difference in how the GPT and Claude model families interpret grounding data when adjusting risk ratings for threat scenarios. The GPT models showed an eagerness to reduce risk ratings based on grounding data, using that contextual information as ironclad evidence that threats had been mitigated (if not outright neutralized). Many of the GPT-produced threat models were marked with language such as "gap resolved," "invalidated," and "new evidence confirms." There was even an instance (Trial 2 of the GPT hybrid trials) where the LLMs strayed from their instructions to rate everything as either low, moderate, high, or critical, instead assigning multiple threat scenarios a risk rating of "None." The GPT family produced only two threat scenarios rated critical across 30 total trials, both from GPT-5 Mini. In a real-world setting, it would be risky to take the GPT family's assertions that controls are fully implemented and effective without question. Although the GPT models identified baseline threats at a comparable percentage to the Claude models in this study, this systematic underrating of threats could ultimately prove disastrous in a production environment where severity drives remediation prioritization and resourcing.

In contrast, the Claude family of models tended to weigh the grounding data more conservatively, acknowledging deployed security controls while preserving high and critical risk ratings for residual risk. These models appeared to prefer assigning risk based on what they could ascertain from the design document, rather than assuming the product would fully implement those controls.

3.3.5. Novel Threat Quality and Reproducibility

The quality and reproducibility of legitimate novel threat findings varied meaningfully across models. In particular, five novel threat concepts were common enough across all models that they could be considered architecturally significant:

1. Feature flag access control
2. Outbox table retention and cleanup
3. Event payload integrity

4. Circuit breaker and back pressure absence
5. Dead letter queue absence

Each of these was a legitimate threat that was omitted from the baseline threat model. Sonnet demonstrated the broadest and most consistent reproducibility of novel threat findings, identifying both feature-flag access-control and outbox-table retention and cleanup threats in 90% of its trials. Haiku also showed a similar pattern of reproducibility, albeit at a lower rate than Sonnet. The GPT models also demonstrated a clear ability to identify and reproduce novel threat concepts, although they differed from the novel findings identified by the Claude models. For example, GPT-5 identified App Mesh/mTLS enrollment gaps in all ten of its trials, whereas the GPT hybrid trials all contained outbox tampering threat scenarios.

The presence of highly reproducible, legitimate, architecturally significant novel threat concepts across two distinct model families strengthens the case for generative AI's role in assisting human security practitioners in the threat modeling process. These are real security concerns grounded in a deep understanding of the architecture presented in the design document, and the fact that multiple independent models identified several common, novel threat concepts provides cross-validation that boosts confidence in the legitimacy of the findings.

4. Implications and Further Research

The most important takeaway from this research is that LLM-assisted threat modeling is viable today. The highest-performing model (Claude 4.5 Sonnet) achieved 68% baseline threat coverage while also surfacing numerous legitimate novel threat scenarios. Even the lowest-performing standalone model (GPT-5) achieved baseline threat coverage of 51% in just under 71 seconds, a major accomplishment that would significantly reduce the time a human security practitioner would need to complete a more comprehensive threat assessment. Of course, none of the LLMs in this study achieved complete threat coverage, whether working on their own or in tandem with

other LLMs, underscoring the need for a human-in-the-loop process to review and validate LLM-generated threat models.

The differences in severity distributions across LLM families were a significant factor in assessing the overall quality of each model's output. Simply put, a model that consistently underrates the severity of various threat scenarios (regardless of how many threats it identifies) is not going to save human security practitioners much in terms of operational costs, due to the need for humans to review and recalibrate those outputs to more appropriately reflect risk. One shortfall of this study was the lack of a consistent rubric when instructing models to adjust risk ratings. While the grounding data explicitly pointed to numerical CVSS scores to determine threat severity, the models were instructed to assign threat scenarios more broadly qualitative severities (low, moderate, high, and critical). Tightening this gap and being more deliberate about risk rating calibration in threat modeling prompts could help improve the GPT family's tendency to underrate threat severity in future studies.

Future studies could also implement alternative methods for RAG. In this study, RAG was achieved by using a vector-based search to surface relevant grounding data for the models. Future research could use alternative search approaches, such as a hybrid approach combining keyword and semantic retrieval. Additionally, future studies could assess the impact on threat severity calibration by varying the volume and specificity of grounding data made available to LLMs.

Overall, the hybrid trials did not produce beneficial results. Both hybrid trial sets showed higher output variance and lower baseline threat coverage than the standalone model trials, despite the theoretical benefits of using a fast model for initial threat model generation before iterating with a deeper model. This is perhaps due to the unintentional introduction of interpretation variance when bringing in the deeper model for threat model refinement, leading to the deeper model reframing existing findings in ways that drift further from alignment with the baseline threat model. This does not mean there is no promise for a hybrid approach to threat modeling. However, future studies that seek to use this approach should consider alternative methods to the sequential generation and refinement method used in this study.

5. Conclusion

This study demonstrates that LLMs can serve as effective tools for threat modeling technical design documents, particularly when provided with sufficient grounding data about the deployment environment. Across 60 trials spanning 6 different model configurations, every model tested identified a meaningful proportion of baseline threats and novel threat scenarios not present in the baseline.

Claude 4.5 Sonnet was the strongest model in terms of accuracy and completeness, with the highest baseline threat coverage, the strongest threat severity ratings, and the most consistent generation of reproducible novel threat scenarios, albeit at the highest runtime cost. Claude 4.5 Haiku offered the best balance of output quality and runtime cost, performing nearly as well on baseline threat coverage while taking only a third of Sonnet's time to run and delivering a respectable risk rating calibration. The GPT family of models demonstrated strong threat identification capacity but also showed a systematic tendency to understate threat severity. Further study is warranted in this area to determine its impact on the real-world prospects of using these LLMs as key components of autonomous threat assessment pipelines.

This study also highlighted clear limitations in all models' capacity for negative pattern recognition, emphasizing the need for meaningful human-in-the-loop reviews of all outputs. Hybrid model configurations yielded poorer results than initially hoped, suggesting that the sequential generation and revision approach used in this study introduces too high a degree of variance (and thus, operational cost) for this type of pipeline to be useful.

One of the most encouraging observations of this study was that each model consistently identified legitimate, architecturally significant novel threats beyond those included in the baseline threat model. This further supports the assertion that LLMs can play a viable supplemental role to human security practitioners in the threat modeling process. It strengthens the case for LLM-assisted threat modeling as a complement to human expertise and experience. Even with less accurate models, this study demonstrates

LLMs' capability to provide a useful first draft of a threat model for a human to review and iterate on, saving significant time and operational costs.

Organizations can take steps now to incorporate LLMs into their existing threat modeling pipelines and security review processes, using them to enhance human productivity. The fundamental capability of LLM-generated threat models has been established, even with variance across models in risk rating calibration, the ability to find and reproduce novel threats, and the production of consistent output. With regular advances in model logic and reasoning, prompt engineering techniques, and grounded data retrieval strategies, the gap between human-created and LLM-generated threat models will continue to narrow.

6. Acknowledgments

The author would like to acknowledge Sachin Kediya for providing access to some of the references cited in this paper.

References

- Al-Hashimi, H.A., Khan, R.A., Alwageed, H.S., Algarni, A.M., Ayoumi, S., & Almagrabi, A.O. (2025). Exploring the role of generative AI in enhancing cybersecurity in software development life cycle. *Array*, 28, 100509. <https://doi.org/10.1016/j.array.2025.100509>
- Alzahrani, A., & Khan, R. A. (2024). Secure software design evaluation and decision making model for ubiquitous computing: A two-stage ann-fuzzy ahp approach. *Computers in Human Behavior*, 153, 108109. <https://doi.org/10.1016/j.chb.2023.108109>
- Braiterman, Z., Shostack, A., Marcil, J., de Vries, S., Michlin, I., Wuyts, K., Hurlbut, R., Schoenfield, B. S. E., Scott, F., Coles, M., Romeo, C., Miller, A., Tarandach, I., Douglén, A., & French, M. (2020). *Threat modeling manifesto*. Threat Modeling Manifesto. <https://threatmodelingmanifesto.org>
- Conklin, Larry. “Threat Modeling Process.” *OWASP Foundation*, https://owasp.org/www-community/Threat_Modeling_Process
- Elsharef, I., Zeng, Z., & Gu, Z. (2024). Facilitating threat modeling by leveraging large language models. *Proceedings 2024 Workshop on AI Systems with Confidential Computing*. <https://doi.org/10.14722/aiscc.2024.23016>
- Ferrag, M. A., Alwahedi, F., Battah, A., Cherif, B., Mechri, A., Tihanyi, N., Bisztray, T., & Debbah, M. (2025). Generative AI in Cybersecurity: A comprehensive review of LLM applications and vulnerabilities. *Internet of Things and Cyber-Physical Systems*, 5, 1–46. <https://doi.org/10.1016/j.iotcps.2025.01.001>
- Kaviyil, P. J., McFadden, D., & Lennon, R. (2023). ChatGPT created threat model. In *2023 Cyber Research Conference (Cyber-RCI)*. IEEE. <https://doi.org/10.1109/Cyber-RCI59474.2023.10671398>

- Mirtaheri, S. L., Movahed, N., Shahbazian, R., Pascucci, V., & Pugliese, A. (2026). Cybersecurity in the age of Generative AI: A systematic taxonomy of AI-powered Vulnerability Assessment and risk management. *Future Generation Computer Systems*, 175, 108107. <https://doi.org/10.1016/j.future.2025.108107>
- Munshi, S., Pathak, S., Ghatode, S., Priyadarshini, T., Chandramouleeswaran, D., & Rana, A. (2025, May 24). *ACSE-eval: Can llms threat model real-world cloud infrastructure?*. arXiv.org. <https://arxiv.org/abs/2505.11565>
- Raghavan, P. (2025). Matthew Adams on ai threat modeling and stride GPT. *IEEE Software*, 42(3), 157–160. <https://doi.org/10.1109/ms.2025.3540515>
- Salesforce. (2024, October 31). *What is a prompt builder?* Salesforce Help. <https://help.salesforce.com/s/articleView?id=000372819&type=3>
- Salesforce. (n.d.-a). *Apex Developer Guide*. Salesforce Developers. https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_dev_guide.htm
- Salesforce. (n.d.-b). *Automate Your Business Processes with Salesforce Flow*. Salesforce Help. https://help.salesforce.com/s/articleView?id=platform.automate_with_salesforce_flow.htm&type=5
- Salesforce. (n.d.-c). *Get started with Data 360 development*. Salesforce Developers. <https://developer.salesforce.com/docs/data/data-cloud-dev/guide/dc-get-started.html>
- Sleegers, W., & Elsey, J. (2026, February 4). *Adoption of LLMS among U.S. Tech Workers*. Rethink Priorities. <https://rethinkpriorities.org/research-area/adoption-llms-tech-workers/>

- Wu, T., Yang, S., Liu, S., Nguyen, D., Jang, S., & Abuadbbba, A. (2025, May 14). *Threatmodeling-LLM: Automating threat modeling using large language models for banking system*. arXiv.org. <https://arxiv.org/abs/2411.17058>
- Yskout, K., Heyman, T., Van Landuyt, D., Sion, L., Wuyts, K., & Joosen, W. (2020). Threat modeling: from infancy to maturity. *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results*, 9–12. <https://doi.org/10.1145/3377816.3381741>
- Zhang, J., Bu, H., Wen, H., Liu, Y., Fei, H., Xi, R., Li, L., Yang, Y., Zhu, H., & Meng, D. (2025). When LLMs meet cybersecurity: A systematic literature review. *Cybersecurity*, 8(1). <https://doi.org/10.1186/s42400-025-00361-w>

Appendix

All materials referenced below are available at: <https://github.com/moswald-sans/llm-threat-modeling-research-files>

A.1. Technical Design Document

This mock technical design document describes an asynchronous order orchestration system deployed on AWS. This document served as the primary subject of analysis for all threat modeling trials.

A.2. Baseline Threat Model

The baseline STRIDE threat model was created by a human security professional prior to running any LLM threat modeling trials. It contains 16 threat scenarios and serves as the reference point for evaluating threat coverage for each trial.

A.3. Grounding Documents (Documents A, B, and C)

These three supplementary documents describe security controls, policies, and architectural features of the deployment environment that are not explicitly covered in the technical design document. Each LLM retrieved chunks of these documents from Salesforce Data Cloud via retrieval augmented generation (RAG) to ground its threat model output.

A.4. Trial Prompts

These were the prompts used in each trial and are presented in the order in which they were used.